



Arbeiten mit Makros

Gültig ab

WinLine Edition 2022 - Version 12

Inhaltsverzeichnis

1.	Einleitung	3
2.	Makros	4
2.1.	Anlage und Aufnahme von Makros	4
2.1.1.	Eigenschaften	7
2.1.2.	Methoden	9
2.1.3.	Ereignisse.....	28
2.2.	Verwaltung von Makros	28
2.3.	Was kann mit Makros gemacht werden?.....	29
2.3.1.	Starten der Makros aus den Favoriten	29
2.3.2.	Starten des Makros mit dem Programm.....	30
2.3.3.	Starten des Makros aus den externen Programmen.....	31
2.3.4.	Starten des Makros aus dem Cockpit.....	31

1. Einleitung

In der WinLine ist die VB-Script-Engine eingebaut, die in verschiedenen Teilbereichen unterschiedliche Funktionen ausführen kann:

- ☐ WinLine FIBU
Formelsprache für das Berechnen von Beträgen in Buchungsarten.
- ☐ WinLine FAKT
Formelsprache für die Abarbeitung von Formeln in Zusammenhang mit Artikelgruppen (Frachtkostenberechnung, Transportversicherung etc. - ist im Standard möglich).
- ☐ WinLine LOHN
Formelsprache für die Abarbeitung von Lohnarten (ist im Standard möglich).
- ☐ WinLine ANBU
Formelsprache für die Berechnung der Staffel-AfA
- ☐ Makros
Aufzeichnen und Abspielen von immer wiederkehrenden Ereignissen (durchführen von Abschlussarbeiten, Datensicherung etc. - ist im Standard möglich)
- ☐ System Skripten
Programmaufrufe oder Ereignisse können unabhängig vom gerade aktiven Fenster ausgeführt werden (WinLine - unabhängige Auswertungen etc. - nur mit eigener Lizenz möglich).
- ☐ Fenster Skripten
Aktionen, die an ein bestimmtes Fenster und dort an ein bestimmtes Ereignis (Anklicken eines Buttons) geknüpft sind - nur mit eigener Lizenz möglich.

2. Makros

Makros sind die Zusammenstellung von einzelnen Ereignissen, die immer den gleichen Ablauf haben und die auch immer die gleichen Einstellungen erfordern.

Beispiele für Makros sind:

- ☐ Tägliche Datensicherung
 - ☐ monatliche Auswertungen
 - ☐ Durchführung Zahlungsverkehr
- etc.

2.1. Anlage und Aufnahme von Makros

Makros können auf zwei Arten angelegt werden:

- ☐ Über einen eigenen Menüpunkt
- ☐ Über den Ribbon

Die Anlage über den Ribbon "Info Center und Makros" ist die einfachere Methode, da über den Menüpunkt die Aufnahme nicht gestartet werden kann.

Der Ribbon "Info Center und Makros"



bietet folgende Funktionen:

➤ Eingabefeld

Das Eingabefeld bietet zwei Möglichkeiten

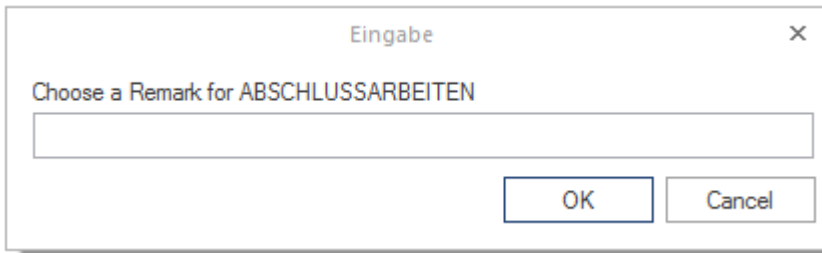
- ☐ Auswahl eines bestehenden Makros
Aus der Auswahllistbox kann ein bereits vorhandenes Makro ausgewählt werden, das dann zur weiteren Bearbeitung zur Verfügung steht.
- ☐ Anlage eines neuen Makros
Ein neues Makro kann angelegt werden, indem im Eingabefeld der Name des neuen Makros eingetragen wird. Wird diese Eingabe bestätigt, erhalten Sie die Abfrage, ob ein neues Makro angelegt werden soll.

Nachdem ein Makro ausgewählt wurde, können zwei Buttons ausgewählt werden:

➤ Makro Aufzeichnen

Wird dieser Button angeklickt, wird die Aufzeichnung des Makros gestartet. Es wird jede Aktion, die innerhalb der WinLine ausgeführt wird, aufgezeichnet.

Nach Starten der Aufzeichnung kann eine Beschreibung des Makros eingegeben werden.

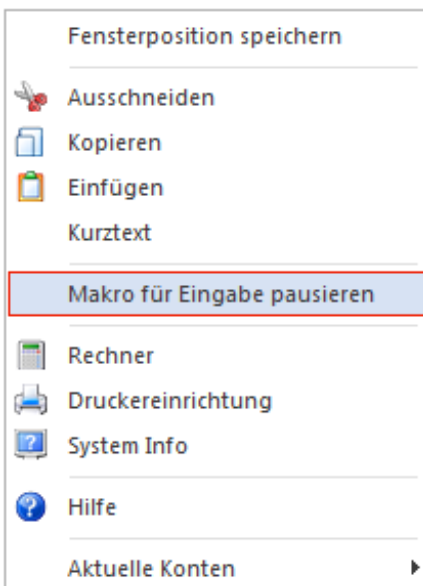


Welche Aktionen können aufgezeichnet werden?

Grundsätzlich werden alle Tastatureingaben aufgezeichnet. Dazu wird auch jeder Mausklick (z.B. auf den Despoolen-Button oder auf den OK-Button etc.) aufgezeichnet.

Innerhalb des Makros können aber auch Eingaben durchgeführt werden. Dazu muss während der Aufnahme im gewünschten Feld die rechte Maustaste geklickt und dann die Option

➤ Pause Macro for Input



gewählt werden. Diese Option ist aber nur in Eingabefeldern möglich und kann innerhalb einer Tabelle (z.B. "Buchen Dialog Stapel" oder "Belegerfassung") nicht verwendet werden.

➤ Makro wiedergeben

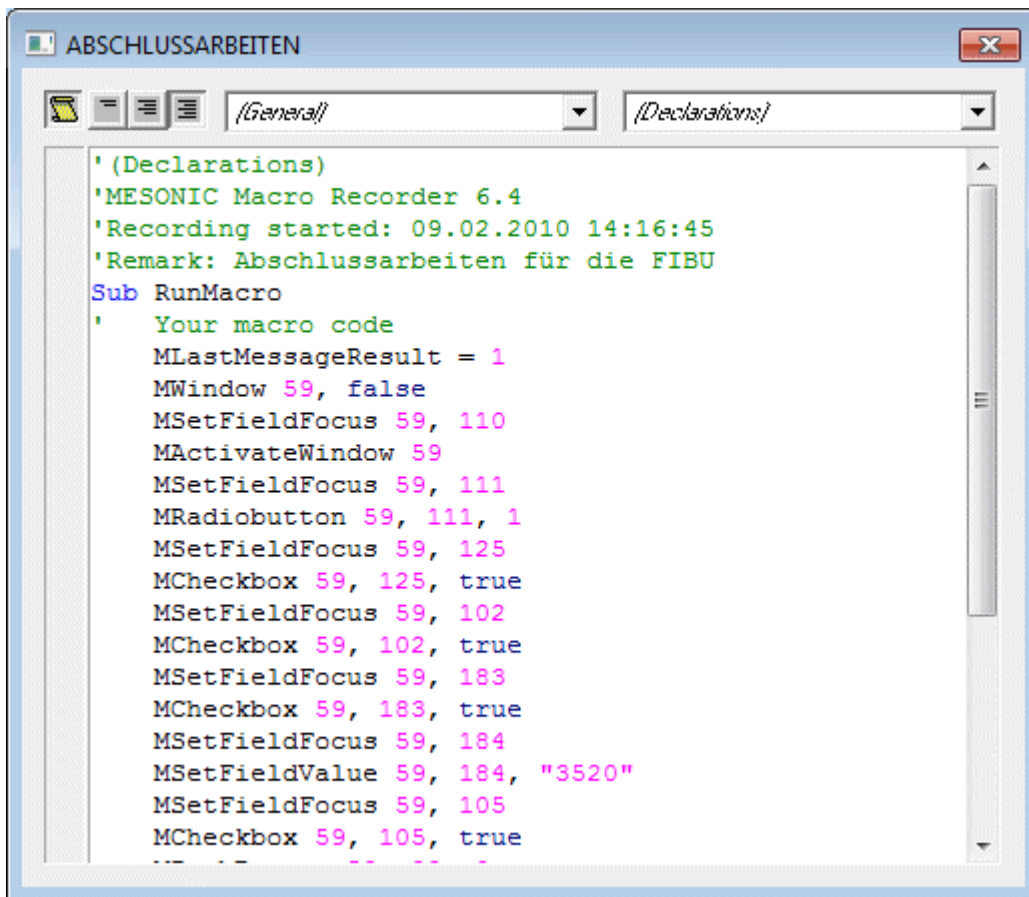
Wird dieser Button angeklickt, wird das eingegebene Makro "abgespielt", wobei alle Aktionen wie bei der Aufnahme durchgeführt werden.

Wurde im Makro die Option "Pause Macro for Input" verwendet, dann bleibt das Makro im entsprechenden Eingabefeld stehen und es kann die erforderliche Eingabe durchgeführt werden. In diesem Fall wird auch der Pause-Button aktiviert. Das Makro wird durch Drücken der F11-Taste (diese Information wird auch beim entsprechendem Eingabefeld angezeigt) bzw. durch Anklicken des Pause-Buttons fortgesetzt.

Wurde eine Makroaufnahme gestartet, steht der Stop-Button zur Verfügung. Damit kann die Aufnahme des Makros gestoppt werden. Beim Abspielen von Makros hat der Button nur dann eine Funktion, wenn eine Eingabe gefordert wird.

Wurde bei einer Aufnahme der Stop-Button angeklickt, wird anschließend das Fenster dargestellt, in dem das Makro in VB-Script angezeigt wird.

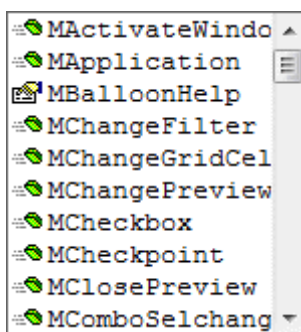
In diesem Fenster können auch noch notwendige Änderungen vorgenommen werden, wobei darauf zu achten ist, dass diese Änderungen so vorzunehmen sind, dass der Ablauf des Programms nicht beeinträchtigt wird.



Im Folgendem finden Sie eine Aufstellung aller Eigenschaften und Methoden, die die WinLine - Makros zulassen. Zusätzlich können alle VB-Script Befehle verwendet.

Um eine Liste aller Funktionen der WinLine im Formelfenster angezeigt zu bekommen, muss nur ein . eingetragen werden.

Dadurch wird das Feld angezeigt. Wird dieses angewählt, dann wird das Wort "CWLMacro" in das Fenster übernommen. Wenn danach nochmals ein . eingegeben wird, wird eine Listbox mit allen Funktionen angezeigt - es kann die gewünschte Funktion ausgewählt werden.



Wenn man sich durch Drücken der Pfeil-nach-Unten-Taste durch die einzelnen Einträge der Listbox bewegt, wird die Verwendung der aktuellen Funktion in einem eigenen Fenster

"Function MApplication(ApplicationNr As Integer) Long" angezeigt. Dadurch lässt sich erkennen, ob man die Funktion mit Parametern aufrufen muss oder nicht.

Wenn vor dem Eintrag in der Listbox das Symbol  angezeigt wird, handelt es sich um eine Funktion.
Wenn vor dem Eintrag in der Listbox das Symbol  angezeigt wird, handelt es sich um eine Variable.

2.1.1. Eigenschaften

BSTR Mname

(readonly)

Enthält den Namen des Makros.

short MLastMessageResult;

(readonly)

Wird bei der Aufzeichnung verwendet und enthält jeweils das Resultat der während der Aufzeichnung ausgelösten Abfragen (z.B.: Wollen Sie speichern ? (JA/NEIN)). Beim Abspielen des Makros wird dieser Wert verwendet, damit das Abspielen nicht durch eine Abfrage unterbrochen wird. Die Variable enthält 0 wenn keine Bildschirmmeldung aufgezeichnet wurde.

VARIANT_BOOL MPrintToArchive;

(read/write)

Entspricht dem Archiv - Button in der Toolbar Leiste. Wird die Eigenschaft auf "TRUE" gesetzt, wird auch der entsprechende Button gedrückt.

VARIANT_BOOL MPrintToSpool;

(read/write)

Entspricht dem Spooler/Drucker - Button in der Toolbar Leiste. Wird die Eigenschaft auf "TRUE" gesetzt, wird auch der entsprechende Button gedrückt.

VARIANT_BOOL MBalloonHelp;

(read/write)

Entspricht dem Aktive Hilfe - Button in der Toolbar Leiste. Wird die Eigenschaft auf "TRUE" gesetzt, wird auch der entsprechende Button gedrückt.

VARIANT_BOOL MSilentMode;

(read/write)

Wird diese Eigenschaft auf "TRUE" gesetzt, dann erfolgt beim Abspielen des Makros keine sichtbare Rückmeldung, erst wenn der Modus zurückgesetzt wird, oder das Makro beendet wird, wird der Bildschirm wieder neu aufgebaut.

VARIANT MParameters;

(read only)

Diese Eigenschaft enthält ein Feld mit den an das Makro übergebenen Parametern. Im Normalfall stehen hier die Systemvariablen 1 bis 19 zur Verfügung, die die Werte des aktuell ausgewählten Mandanten wieder spiegeln.

Makros, die aus Formularen heraus (über Hyperlinks) gestartet werden, haben den Inhalt des Hyperlinkfeldes als ersten Applikationsparameter im Feld an Position 20.

Beim Starten von externen Applikationen können ebenfalls Makros hinterlegt werden, dort können zusätzliche Parameter an das Makro übergeben werden, die dann ebenfalls ab Position 20 im Feld stehen.

Damit die Feldwerte im Makro verwendet werden können, müssen sie zuerst an eine Variable zugewiesen werden.

Beispiel

Im folgenden Beispiel werden im Makro die Systemvariablen in einer Bildschirmmeldung ausgegeben:

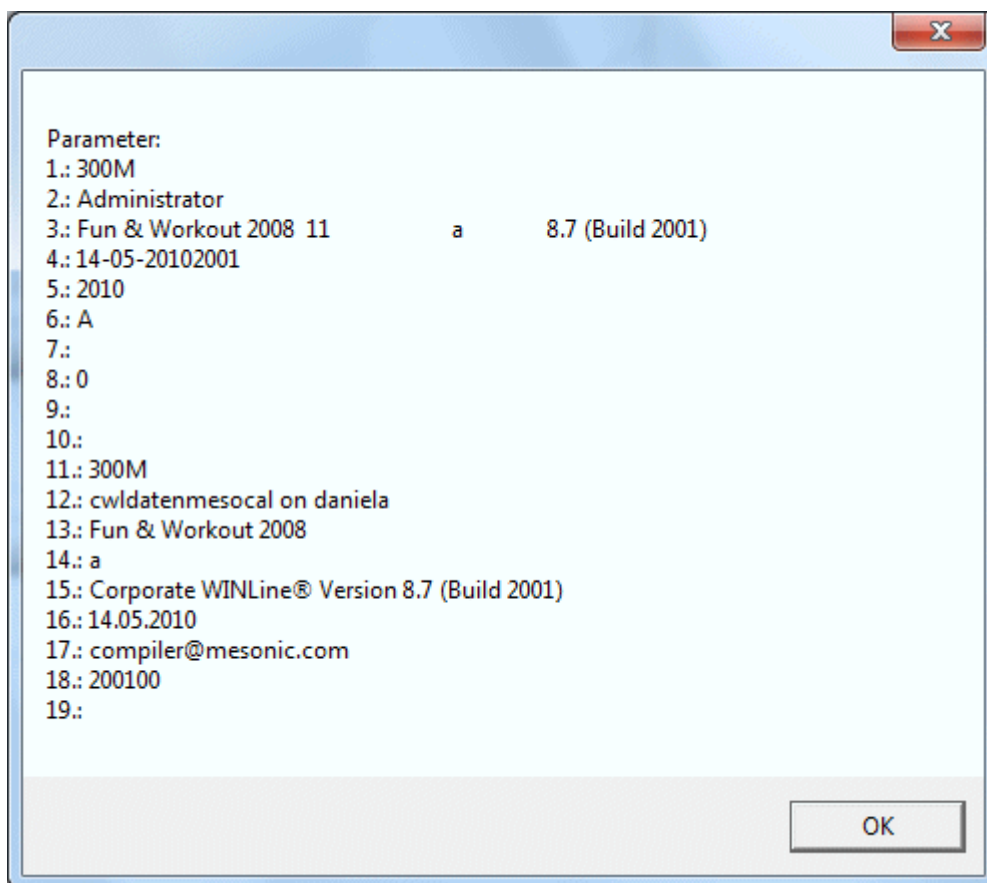
```
Sub RunMacro
    ' Your macro code
    params = MParameters ' unbedingt notwendig um auf die Parameter
                        ' als Feld zugreifen zu können

    CRLF = chr(13)&chr(10)
    msg = "Parameter:" & CRLF

    ' Der Feldwert 0 ist immer leer deshalb beginnt
    ' die Schleife bei 1
    For i = 1 To ubound(params)
        msg = msg & i & ".: " & params(i) & CRLF
    Next

    ' gefundene Parameter anzeigen
    msgbox msg

End Sub
```



Im Programm WinLine START, im Menüpunkt



Applikationen



Externe Programme...

können auch Makros eingetragen werden. Zusätzlich zum Makronamen können dort zusätzliche Parameter angegeben werden:

MACRO:MACRONAME {Art:Wert}{Art:Wert}.... usw.

WinLine Makros

mesonic © 07/2020

wobei Art

- ☐ - **CompanyValue** (Wert aus dem Mandantenstamm = Spaltennummer) oder
- ☐ - **Constant** (Wert ist eine beliebige Zeichenkette)

sein kann.

Beispiel:

MACRO:TEST {CompanyValue:1}{CompanyValue:2}{CompanyValue:3}

MACRO:TEST {Constant:Ein Test}{Constant:100,20}

short MCurrentPeriod;

(readonly)

In dieser Eigenschaft steht immer die aktuelle Periode (das aktuelle Monat).

2.1.2. Methoden

void MWait(long lMilliseconds);

Das Abspielen des Makros wird für die angegebene Zeitspanne angehalten.

➤ **Parameter:**

lMilliseconds Anzahl der Millisekunden, die das Makro warten soll

➤ **Rückgabe:**

keine

VARIANT BOOL MSetFieldFocus(short nWinId, short nFieldId);

Setzt den Cursor auf das angegebene Feld. Ist mit einem Mausklick des Anwenders zu vergleichen.

➤ **Parameter:**

nWinId Nummer des Fensters, in dem sich das gewünschte Feld befindet

nFieldId Nummer des Feldes, das den Focus erhalten soll

➤ **Rückgabe:**

True/False, abhängig davon ob der Focus versetzt werden konnte

void MSetFieldValue(short nWinId, short nFieldId, BSTR strValue);

Setzt den Wert des angegebenen Feldes. Hat denselben Effekt als ob der Benutzer in dem Feld den Text eingibt und danach die ENTER Taste drückt.

➤ **Parameter:**

nWinId Nummer des Fensters in dem das gewünschte Feld sich befindet

nFieldId Nummer des Feldes, das den Wert erhalten soll

➤ **Rückgabe:**

keine

BSTR MGetFieldValue(short nWinId, short nFieldId);

Gibt den Wert des angegebenen Feldes als Text zurück.

➤ **Parameter:**

nWinId Nummer des Fensters in dem das gewünschte Feld sich befindet

nFieldId Nummer des Feldes, von dem der Wert geholt werden soll

➤ **Rückgabe:**

Der Feldinhalt als Text

void MSetGridValue(short nWinId, short nFieldId, short nRow, short nColumn, BSTR strValue);

Setzt den Cursor in der Tabelle auf die angegebene Zeile/Spalte und fügt dort den Text ein.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Tabelle befindet
 nFieldId Nummer der Tabelle
 nRow die aktuelle Zeile in der Tabelle
 nColumn die aktuelle Spalte in der Tabelle
 strValue der Text, der in der Zelle eingefügt werden soll

➤ **Rückgabe:**

keine

long MPushButton(short nWinId, short nButtonId, long lParam);

Der angegebene Button wird gedrückt. Der Parameter **lParam** kann auf 0 belassen werden. Bei der Aufnahme des Makros könnte der Wert allerdings auf ungleich 0 stehen (Beim Verlassen der Applikation erhält der Ende - Button hier einen speziellen Wert, damit das Fenster erkennen kann, dass es in Folge des Programmendes geschlossen wird).

➤ **Parameter:**

nWinId Nummer des Fensters in dem der Button sich befindet
 nButtonId Nummer des Buttons, der gedrückt werden soll
 lParam im Standardfall 0

➤ **Rückgabe:**

Nicht definiert

long MGridMatchCode(short nWinId, short nFieldId, short nRow, short nColumn, BSTR strSearchText, BOOL bExtended);

Entspricht dem Klick auf die Lupe in einer Tabelle (bzw. der <F9> Taste).

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Tabelle befindet
 nFieldId Nummer der Tabelle
 nRow die aktuelle Zeile in der Tabelle
 nColumn die aktuelle Spalte in der Tabelle
 strSearchText der Text den das Editfeld enthielt als <F9> gedrückt wurde
 bExtended ist TRUE falls auf die Lupe geklickt wurde, bzw. <SHIFT><F9> gedrückt wurde

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MTreeExpand(short nWinId, short nFieldId, long lParam, long lTreeData);

Dieses Kommando wird aufgezeichnet, wenn in einer Baumstruktur auf das Symbol  geklickt wurde, sodass der dort enthaltene Zweig aufgeklappt wird. Das Kommando sollte nicht manuell eingefügt werden, weil die Daten in **lParam** und **lTreeData** nur bei der Aufzeichnung richtig gesetzt werden können.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Bauelement befindet

nFieldId Nummer des Baumelements
lParam (nur für interne Zwecke)
lTreeData (nur für interne Zwecke)

➤ **Rückgabe:**

1 falls kein Fehler auftrat, sonst 0

long MTreeCollapse(short nWinId, short nFieldId, long lParam, long lTreeData);

Dieses Kommando wird aufgezeichnet, wenn ein aufgeklappter Baum in einer Baumstruktur vom Benutzer geschlossen wird (anklicken des  Symbols). Das Kommando sollte nicht manuell eingefügt werden, weil die Daten in `lParam` und `lTreeData` nur bei der Aufzeichnung richtig gesetzt werden können.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Bauelement befindet
nFieldId Nummer des Baumelements
lParam (nur für interne Zwecke)
lTreeData (nur für interne Zwecke)

➤ **Rückgabe:**

1 falls kein Fehler auftrat, sonst 0

long MTreeSelChange(short nWinId, short nFieldId, long lParam, long lTreeData);

Dieses Kommando wird aufgezeichnet, wenn der Benutzer in einer Baumstruktur einen Eintrag auswählt. Das Kommando sollte nicht manuell eingefügt werden, weil die Daten in `lParam` und `lTreeData` nur bei der Aufzeichnung richtig gesetzt werden können.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Bauelement befindet
nFieldId Nummer des Baumelements
lParam (nur für interne Zwecke)
lTreeData (nur für interne Zwecke)

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MTreeDelete(short nWinId, short nFieldId, long lParam, long lTreeData);

Dieses Kommando wird aufgezeichnet, wenn der Benutzer in einer Baumstruktur einen Eintrag löscht. Das Kommando sollte nicht manuell eingefügt werden, weil die Daten in `lParam` und `lTreeData` nur bei der Aufzeichnung richtig gesetzt werden können.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Bauelement befindet
nFieldId Nummer des Baumelements
lParam (nur für interne Zwecke)
lTreeData (nur für interne Zwecke)

➤ **Rückgabe:**

1 falls kein Fehler auftrat, sonst 0

long MTreeDbClick(short nWinId, short nFieldId, long lParam, BSTR strItemText);

Dieses Kommando wird aufgezeichnet, wenn der Benutzer in einer Baumstruktur einen Eintrag doppelt anklickt. Das Kommando sollte nicht manuell eingefügt werden, weil die Daten in `lParam` nur bei der Auf-

zeichnung richtig gesetzt werden können. Der Parameter `strItemText` enthält den Text des selektierten Items.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Bauelement befindet
 nFieldId Nummer des Bauelements
 lParam (nur für interne Zwecke)
 strItemText der Text des gewählten Elements

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MMatchCode(short nWinId, short nFieldId, long lParam, BSTR strSearchText, VARIANT BOOL bExtended);

Entspricht dem Anklicken der Lupe bei einem Eingabefeld bzw. dem Drücken der F9-Taste in einem Eingabefeld.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Eingabefeld befindet
 nFieldId Nummer des Eingabefeldes
 lParam immer 0
 strSearchText der Text den das Eingabefeld enthielt als F9 gedrückt wurde
 bExtended ist "TRUE" falls auf die Lupe geklickt wurde, bzw. <SHIFT><F9> gedrückt wurde

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MGridLeftClick(short nWinId, short nFieldId, short nRow, short nColumn);

Entspricht dem Klick mit der linken Maustaste auf eine Zelle in einer Tabelle (die Zelle darf kein Eingabefeld, keine Auswahllistbox oder keine Checkbox enthalten). Das Kommando wird aufgezeichnet, wenn der Benutzer die Aktion durchführt. Dies führt beim Abspielen aber nicht zu einem Klick auf die Tabelle, sondern die Applikation reagiert als ob der Klick durchgeführt worden wäre.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Tabelle befindet
 nFieldId Nummer der Tabelle
 nRow die aktuelle Zeile in der Tabelle
 nColumn die aktuelle Spalte in der Tabelle

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

➤ **long MGridRightClick(short nWinId, short nFieldId, short nRow, short nColumn);**

Entspricht dem Klick mit der rechten Maustaste auf eine Zelle in einer Tabelle (die Zelle darf kein Eingabefeld, keine Auswahllistbox oder keine Checkbox enthalten). Das Kommando wird aufgezeichnet, wenn der Benutzer die Aktion durchführt. Dies führt beim Abspielen aber nicht zu einem Klick auf die Tabelle, sondern die Applikation reagiert als ob der Klick durchgeführt worden wäre.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Tabelle befindet
 nFieldId Nummer der Tabelle
 nRow die aktuelle Zeile in der Tabelle
 nColumn die aktuelle Spalte in der Tabelle

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MGridDbClick(short nWinId, short nFieldId, short nRow, short nColumn):

Entspricht dem Doppelklick mit der linken Maustaste auf eine Zelle in einer Tabelle (die Zelle darf kein Eingabefeld, keine Auswahllistbox oder keine Checkbox enthalten). Das Kommando wird aufgezeichnet, wenn der Benutzer die Aktion durchführt. Dies führt beim Abspielen aber nicht zu einem Klick auf die Tabelle, sondern die Applikation reagiert als ob der Klick durchgeführt worden wäre.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Tabelle befindet
nFieldId	Nummer der Tabelle
nRow	die aktuelle Zeile in der Tabelle
nColumn	die aktuelle Spalte in der Tabelle

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MGridColLeftClick(short nWinId, short nFieldId, short nColumn):

Entspricht dem Klick mit der linken Maustaste auf eine Zelle im Kopf einer Tabelle. Das Kommando wird aufgezeichnet, wenn der Benutzer die Aktion durchführt. Dies führt beim Abspielen aber nicht zu einem Klick auf die Tabelle, sondern die Applikation reagiert als ob der Klick durchgeführt worden wäre.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Tabelle befindet
nFieldId	Nummer der Tabelle
nRow	die aktuelle Zeile in der Tabelle
nColumn	die aktuelle Spalte in der Tabelle

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MGridInfo(short nWinId, short nFieldId, short nRow, short nColumn):

Dieses Kommando wird eingefügt, wenn in einer Tabelle in einem Eingabefeld oder einer Auswahllistbox die Taste <F8> gedrückt wird

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Tabelle befindet
nFieldId	Nummer der Tabelle
nRow	die aktuelle Zeile in der Tabelle
nColumn	die aktuelle Spalte in der Tabelle

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MGridCheckbox(short nWinId, short nFieldId, short nRow, short nColumn, VARIANT BOOL bChecked):

Entspricht dem Klick mit der linken Maustaste auf eine Zelle einer Tabelle die eine Checkbox enthält. Beim Abspielen wird der Focus auf die entsprechende Zelle gesetzt und die Checkbox entsprechend dem Wert in `bChecked` gesetzt.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Tabelle befindet
--------	--

nFieldId	Nummer der Tabelle
nRow	die aktuelle Zeile in der Tabelle
nColumn	die aktuelle Spalte in der Tabelle
bChecked	enthält TRUE falls die Checkbox gecheckt werden soll, andernfalls FALSE

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MGridLeave(short nWinId, short nFieldId, short nToFgId, short nFromRow):

Dieses Kommando wird aufgezeichnet, wenn der Benutzer eine Tabelle verlässt (auf ein anderes Feld im Fenster klickt, oder mit TAB aus der Tabelle springt)

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Tabelle befindet
nFieldId	Nummer der Tabelle
nToFgId	die Nummer des Elements, das nach Verlassen der Tabelle aktiv wurde
nFromRow	die aktuelle Zeile in der Tabelle

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MGridComboSelchange(short nWinId, short nFieldId, short nRow, short nColumn, BSTR strFieldText):

Dieses Kommando wird aufgezeichnet, wenn der Anwender die Selektion in einer Combobox in einer Tabelle ändert. Beim Abspielen wird der Wert in **strFieldText** in die Combobox eingefügt.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Tabelle befindet
nFieldId	Nummer der Tabelle
nRow	die aktuelle Zeile in der Tabelle
nColumn	die aktuelle Spalte in der Tabelle
strFieldText	der selektierte Text aus der Combobox

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MDrillDown(short nWinId, short nFieldId, long lParam):

Dieses Kommando wird aufgezeichnet, wenn der Anwender in einem beliebigen Element in einem Fenster die Taste <F8> drückt, der Parameter **lParam** ist immer 0.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem das aktive Feld ist
nFieldId	Nummer des Feldes, indem <F8> gedrückt wurde
lParam	immer 0

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MListBoxSelChange(short nWinId, short nFieldId, long nIndex):

Dieses Kommando wird aufgezeichnet, wenn der Anwender in einer Listbox die Selektion ändert. In **nIndex** wird der Index des selektierten Eintrags übergeben.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Listbox befindet
nFieldId Nummer der Listbox
nItemIndex Index des gewählten Listbox Elements

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MListBox(short nWinId, short nFieldId, long lItemIndex);

Dieses Kommando wird aufgezeichnet, wenn der Anwender in einer Listbox ein Item mit ENTER oder Doppelklick auswählt. In **nItemIndex** wird der Index des selektierten Eintrags übergeben.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Listbox befindet
nFieldId Nummer der Listbox
nItemIndex Index des gewählten Listbox Elements

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MCheckbox(short nWinId, short nFieldId, BOOL bChecked);

Dieses Kommando wird aufgezeichnet, wenn der Anwender auf eine Checkbox klickt. In **bChecked** wird der Status der Checkbox übergeben.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Checkbox befindet
nFieldId Nummer der Checkbox
nItemIndex Index des gewählten Checkbox Elements

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MRadiobutton(short nWinId, short nFieldId, long lIndex);

Dieses Kommando wird aufgezeichnet, wenn der Anwender auf einen Radiobutton klickt. In **lIndex** wird der Index des gewählten Button übergeben (Radiobuttons sind in Gruppen organisiert wobei jeder Button in der Gruppe einen Index beginnend bei 0 hat)

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich der Radiobutton befindet
nFieldId Nummer des Radiobutton
nItemIndex Index des gewählten Radiobutton Elements

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MComboSelchange(short nWinId, short nFieldId, BSTR strFieldText);

Dieses Kommando wird aufgezeichnet, wenn der Anwender die Selektion in einer Combobox in einem Fenster ändert. Beim Abspielen wird der Wert in **strFieldText** in die Combobox eingefügt.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Combobox befindet
nFieldId Nummer der Combobox
strFieldText der Text des selektierten Combobox Eintrags

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MChangeGridCell (short nWinId, short nFieldId, long lParam):

Dieses Kommando wird aufgezeichnet, wenn der Anwender sich in der Tabelle bewegt (entweder über die Tastatur oder mit der Maus).

lParam beschreibt welche Methode der Anwender verwendet hat. Wurde die Tastatur verwendet, dann enthält dieser Parameter die Nummer der verwendeten Taste (13 = ENTER, 39 = Cursor Rechts, 37 = Cursor links, 40 = Cursor hinab, 38 = Cursor hinauf, 35 = Cursor Ende, 36 = Cursor Pos1, 33 = Cursor Bild hinauf, 34 = Cursor Bild hinunter). Wurde mit der Maus geklickt, dann enthält **lParam** die Zeile/Spalte der Zelle, wobei die Zeile in den unteren 16 Bit kodiert ist, die Spalte in den oberen 16 Bit.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Tabelle befindet
 nFieldId Nummer der Tabelle
 lParam abhängig von der Art, wie die Zelle aktiviert wurde, entweder ein Tastencode oder ein zusammengesetzter Wert aus Zeile und Spalte
 nColumn die aktuelle Spalte in der Tabelle

➤ **Rückgabe:**

1 falls kein Fehler auftrat, sonst 0

long MWindow (short nWinId, VARIANT_BOOL bQuiet, VARIANT Param):

Dieses Kommando wird aufgezeichnet, wenn der Anwender ein Fenster öffnet oder zwischen verschiedenen Fenstern hin und her wechselt. Der Parameter **bQuiet** wird nur dann auf "TRUE" gesetzt, wenn das Fenster neu geöffnet wird, dabei aber nicht angezeigt werden soll. Der Parameter ist normalerweise immer FALSE. Der Parameter **Param** wird beim Aufzeichnen eines Makros nie erzeugt. Er kann verwendet werden, wenn der Aufruf selbst programmiert wird und das aufgerufene Fenster den zusätzlichen Parameter optional für bestimmte Funktionen verwendet.

➤ **Parameter:**

nWinId Nummer des Fensters das geöffnet/aktiviert werden soll
 bQuiet TRUE falls ein neu geöffnetes Fenster nicht sichtbar sein soll, normalerweise FALSE
 Param optionaler Parameter für den Fensteraufruf

➤ **Rückgabe:**

1 falls das Fenster geöffnet/aktiviert wurde, sonst 0

MApplication (short ApplicationNr):

Dieses Kommando wird aufgezeichnet, wenn der Anwender zu einer anderen Applikation innerhalb der WinLine wechselt. Der Parameter **ApplicationNr** kann folgende Werte annehmen:

Programm	Nummer
START	0
FIBU	1
FAKT	2
LOHN A	3
LIST	4
KORE	5
ANBU	6
ADMIN	8
EXIM	9
INFO	11

LOHN D	18
PROD	20

➤ **Parameter:**

ApplicationNr Nummer des Applikation

➤ **Rückgabe:**

1 falls zu der Applikation gewechselt werden konnte

MExternalApplication (short nId):

Dieses Kommando wird aufgezeichnet, wenn der Anwender über den Menüpunkt 'Applikationen' eine selbst definierte Applikation startet. Der Parameter **nId** ist der Index der gestarteten Applikation, wobei die erste eingetragene Applikation den Index 0 hat.

➤ **Parameter:**

nId Index der benutzerdefinierten Applikation

➤ **Rückgabe:**

1 falls die Applikation gestartet werden konnte

void MToClipboard ():

Dieses Kommando wird aufgezeichnet, wenn der Anwender in einem Eingabefeld den markierten Inhalt in die Zwischenablage kopiert. Beim Abspielen des Makros wird mit diesem Kommando die gerade aktive Selektion im aktuellen Eingabefeld in die Zwischenablage kopiert.

➤ **Parameter:**

keine

➤ **Rückgabe:**

keine

void MFromClipboard ():

Dieses Kommando wird aufgezeichnet, wenn der Anwender in ein Eingabefeld die Zwischenablage einfügt. Beim Abspielen des Makros wird mit diesem Kommando der zu dem Zeitpunkt des Abspielens vorhandene Zwischenablageinhalt in das gerade aktive Editfeld eingefügt.

➤ **Parameter:**

keine

➤ **Rückgabe:**

keine

void MStop ():

Dieses Kommando wird nicht aufgezeichnet. Es kann vom Anwender in das Makro eingefügt werden, wenn die Makroabarbeitung zu einem bestimmten Zeitpunkt beendet werden soll.

➤ **Parameter:**

keine

➤ **Rückgabe:**

keine

void MPauseForInput (BSTR strNextRoutineAfterPause, short nWinId, short nFieldId);

Dieses Kommando wird aufgezeichnet, wenn der Benutzer in einem Editfeld über die rechte Maustaste den Menüpunkt 'Pause Macro for Input' anwählt. Dies führt beim Ausführen des Makros dazu, dass die Makroabarbeitung genau bei diesem Schritt anhält, um dem Benutzer eine Eingabe zu erlauben. Danach kann das Makro mit der Funktionstaste <F11> weiter abgearbeitet werden. Der Befehl sollte auf keinen Fall manuell eingefügt werden.

➤ Parameter:

strNextRoutineAfterPause	Der Name der Subroutine, die nach der Benutzerunterbrechung zur Fortführung des Makros gestartet werden soll
nWinId	Die Nummer des Fensters in dem die Unterbrechung erfolgt
nFieldId	Die Nummer des Feldes in dem die Unterbrechung erfolgte

➤ Rückgabe:

keine

void MPauseForFilter (BSTR strNextRoutineAfterPause);

Dieses Kommando wird aufgezeichnet, wenn der Benutzer im Filtereingabefenster über die rechte Maustaste den Menüpunkt "Makro für die Filtereingabe pausieren" anwählt. Dies führt beim Ausführen des Makros dazu, dass die Makroabarbeitung während der Eingabe in der Filtereingabe pausiert, um dem Benutzer spezifische Eingaben zu erlauben. Sobald das Fenster mit F5 oder ESC geschlossen wird, wird das Makro weiter ausgeführt (bei OK wird die Auswertung entsprechend den getätigten Filterbedingungen ausgeführt).

➤ Parameter:

strNextRoutineAfterPause	Der Name der Subroutine, die nach der Benutzerunterbrechung zur Fortführung des Makros gestartet werden soll
--------------------------	--

➤ Rückgabe:

keine

void MRunMacro (BSTR MacroName);

Dieses Kommando wird nicht aufgezeichnet. Es kann vom Anwender in das Makro eingefügt werden, um ein weiteres Makro zu starten. Nachdem dieses andere Makro die Abarbeitung beendet hat, läuft das aktuelle Makro nach diesem Kommando weiter.

➤ Parameter:

MacroName	Name des Makros das gestartet werden soll
-----------	---

➤ Rückgabe:

keine

void MRunForm (BSTR MacroName, short bMode);

Dieses Kommando wird nicht aufgezeichnet. Es kann vom Anwender in das Makro eingefügt werden, um ein System Skript zu starten. Mit dem Parameter **bMode** kann der Modus festgelegt werden, wie das System Skript ablaufen soll (0.. als normales Fenster, 1.. als modales Fenster, 2... als applikationsübergreifendes Fenster, das immer im Vordergrund ist)

➤ Parameter:

MacroName	Name des System Skripts das gestartet werden soll
bMode	Modus mit dem das System Skript ablaufen soll
0...	normale (wird beim Umschalten auf andere Appl. versteckt)
1...	modal (erst nach Schließen des Skripts kann in der Appl. weitergearbeitet werden)
2...	applikationsübergreifend (bleibt auch beim Umschalten auf andere Appl. im Vordergrund)

➤ **Rückgabe:**

keine

long MChangeFilter(short nWinId, short nFieldId, BSTR FilterName);

Dieses Kommando wird aufgezeichnet, wenn der Benutzer in einem Fenster mit Filter-Funktion einen Filter aus der Auswahllistbox auswählt. Der Parameter **FilterName** enthält den Namen des gewählten Filters.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich der Filter befindet
nFieldId immer 0
FilterName Name des ausgewählten Filters

➤ **Rückgabe:**

Immer 0

void MLastDialogResult (BOOL bResult, VARIANT value, BSTR Remark);

Dieses Kommando wird aufgezeichnet, wenn im Programm ein Lohn- oder Faktformelfenster während des Makroaufzeichnens geöffnet wird. Das Kommando registriert das Ergebnis des Dialogs um beim Abspielen des Makros das Ergebnis der Aufzeichnung zur Hand zu haben und das Formelfenster nicht mehr geöffnet werden muss.

➤ **Parameter:**

bResult **True** wenn das Formelfenster mit OK beendet wurde, andernfalls **False**
value der Rückgabewert des Formelfensters
Remark wird zur Zeit nicht verwendet

➤ **Rückgabe:**

keine

void MLastReplacedForm (short Number);

Dieses Kommando wird aufgezeichnet, wenn im Programm der Formularersetzen Dialog eingeblendet wird, in dem der Benutzer ein vom Standardformular abweichendes Formular auswählen kann.

➤ **Parameter:**

Number die Nummer des ausgewählten Ersatzformulars

➤ **Rückgabe:**

keine

BOOL MChangePreviewPage (short WinId, short PreviewId, short PageNumber);

Dieses Kommando wird aufgezeichnet, wenn im Programm in einer Druckvorschau die angezeigte Seite geändert wird (d.h. in der Druckvorschau geblättert wird).

➤ **Parameter:**

WinId die Nummer des Druckvorschau Fensters
PreviewId die interne Nummer des Druckvorschau Elements
PageNumber die Seitennummer zu der geblättert wurde

➤ **Rückgabe:**

True/False, abhängig davon ob beim Abspielen des Makros zu der Seite gewechselt werden konnte

BOOL MGridSort (short WinId, short FieldId, short SortColumn1, short SortColumn2, short SortDirection):

Dieses Kommando wird aufgezeichnet, wenn im Programm in einer Tabelle eine Spalte sortiert wird.

➤ **Parameter:**

WinId die Nummer des Fensters
FieldId die interne Nummer des Tabellenelements
SortColumn1 die 1. logische Spaltennummer die sortiert wird
SortColumn2 die 2. logische Spaltennummer die sortiert wird (ist 0, wenn nur nach einer Spalte sortiert wird)
SortDirection 1... aufwärts, 2... abwärts

➤ **Rückgabe:**

True/False, abhängig davon ob das Sortieren beim Abspielen des Makros durchgeführt werden konnte

long MRetChar (short nWinId, short nFieldId, short Key):

Dieses Kommando wird aufgezeichnet, wenn der Anwender in einem Editfeld eine Taste drückt, die von der Applikation speziell behandelt wird.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Editfeld befindet
nFieldId Nummer des Editfeldes
Key die gedrückte Taste

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MMessageF3 (short nWinId, short nFieldId, BOOL bShift):

Dieses Kommando wird aufgezeichnet, wenn der Anwender in einem Editfeld die Taste F3 drückt, die abhängig vom Typ des Editfeldes unterschiedliche Aktionen auslöst.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Editfeld befindet
nFieldId Nummer des Editfeldes
bShift True falls <UMSCHALT> <F3> gedrückt wurde

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

MQuitApplication ()

Dieses Kommando beendet die Applikation. Es wird die Auswahl des Menüpunkts ‚Datei|Ende‘ simuliert. Danach darf und kann kein Macrobefehl mehr kommen, weil die Applikation geschlossen wird und damit die Makroabarbeitung ebenfalls endet.

➤ **Parameter:**

keine

➤ **Rückgabe:**

keine

BOOL MStartExecutable (BSTR Application, BSTR Parameters)

Dieses Kommando startet ein beliebiges Programm.

➤ **Parameter:**

Application Der Name des zu startenden Programmes inclusive Pfad wenn notwendig

Parameters Programmparameter, falls welche notwendig sind.

➤ **Rückgabe:**

True/False Meldet den Erfolgreichen Start der Applikation

BSTR MGetPlainRTFFieldValue (short nWinId, short nFieldId)

Dieses Kommando holt den Text aus einem RTF Feld und konvertiert diesen in reinen ANSI Text (ohne Formatierungen).

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich das Editfeld befindet

nFieldId Nummer des Editfeldes

➤ **Rückgabe:**

Der konvertierte RTF Text.

MClosePreview (short nWinId)

Schließt ein Druckvorschaufenster mit der Fenster Nummer nWinId.

➤ **Parameter:**

nWinId Nummer des Fensters in dem sich die Druckvorschau befindet

➤ **Rückgabe:**

Keine

MActivateWindow (short nWinId)

Aktiviert das Fenster mit der Nummer nWinId. Ist das angegebene Fenster bereits aktiv wird nichts gemacht, andernfalls wird das neue Fenster zum aktiven Fenster.

➤ **Parameter:**

nWinId Nummer des Fensters das aktiviert werden soll

➤ **Rückgabe:**

keine

long MCompanyYearChange (short nWinId, short InternalYearValue);

Bei Fenstern, die in der Werkzeugleiste eine Liste der Wirtschaftsjahre aufweisen (z.B. Kontoblatt), kann mit diesem Befehl das ausgewählte Wirtschaftsjahr verändert werden.

➤ **Parameter:**

nWinId Nummer des Fensters das aktiviert werden soll

InternalYearValue das Wirtschaftsjahr im internen numerischen Format

➤ **Rückgabe:**

Keine

long MTreeCheck(short nWinId, short nFieldId, long lParam);

Dieses Kommando wird aufgezeichnet, wenn der Benutzer eine Baumstruktur mit TAB oder der Maus verlässt. Das Kommando sollte nicht manuell eingefügt werden, weil die Daten in **lParam** nur bei der Aufzeichnung richtig gesetzt werden können.

➤ Parameter:

nWinId Nummer des Fensters in dem sich das Bauelement befindet
nFieldId Nummer des Bauelements
lParam (nur für interne Zwecke)

➤ Rückgabe:

Ist abhängig von der Applikation, normalerweise 1

long MTreeCheckbox(short nWinId, short nFieldId, long lParam);

Dieses Kommando wird aufgezeichnet, wenn der Benutzer in einer Baumstruktur, die Checkboxes enthält, den Inhalt dieser verändert. Das Kommando sollte nicht manuell eingefügt werden, weil die Daten in **lParam** nur bei der Aufzeichnung richtig gesetzt werden können.

➤ Parameter:

nWinId Nummer des Fensters in dem sich das Bauelement befindet
nFieldId Nummer des Bauelements
lParam (nur für interne Zwecke)

➤ Rückgabe:

Ist abhängig von der Applikation, normalerweise 1

void MPrintGrid (short nWinId, short nFieldId);

Mit diesem Befehl wird eine Tabelle ausgedruckt (es wird das gleiche durchgeführt wie bei der Auswahl des Kontextmenübefehls: ‚Tabelle ausdrucken‘).

➤ Parameter:

nWinId Nummer des Fensters in dem sich die Tabelle befindet
nFieldId Nummer der Tabelle

➤ Rückgabe:

keine

long MCompanyChange (BSTR Company, short CompanyYear);

Mit diesem Befehl wird der aktuelle Mandant gewechselt. Der Befehl wird aufgezeichnet, wenn in der Hauptwerkzeugleiste des Programms ein anderer Mandant oder ein anderes Wirtschaftsjahr ausgewählt wird.

➤ Parameter:

Company Die Nummer des Mandanten (z.B. ‚300M‘)
CompanyYear Das gewünschte Wirtschaftsjahr im internen numerischen Format

➤ Rückgabe:

1 bei erfolgreichem Mandantenwechsel, andernfalls 0.

MPrintPreview (short nWinId, short nFieldId, BSTR Printer);

Dieses Kommando wird aufgezeichnet, wenn in einem Druckvorschauenfenster auf den Druckschaltknopf gedrückt wird. Abhängig davon ob in dem darauf folgenden Dialog ein neuer Drucker ausgewählt wird, oder

der Standarddrucker verwendet wird, wird im Parameter PrinterDescription der gewählte Drucker übergeben, oder der Parameter bleibt leer.

➤ **Parameter:**

nWinId	Fenster Nummer des Druckvorschau Fensters
bFieldId	Feldnummer der Druckvorschau
Printer	falls ein Drucker ausgewählt wurde, die Druckerbeschreibung, andernfalls leer (Standarddrucker)

➤ **Rückgabe:**

keine

long MMessageDynamicMenuCommand (short nWinId, short nFieldId, short CommandIndex):

Das Kommando wird aufgezeichnet, wenn der Benutzer in der Werkzeugleiste eines Fensters eine Schaltfläche mit integrierten Menü auswählt. Dieser Schaltflächentyp wird in der aktuellen Programmversion für die Ausgabe einer Auswertung auf verschiedene Zielmedien verwendet (z.B: Ausgabe auf Drucker oder Bildschirm).

➤ **Parameter:**

nWinId	Nummer des Fensters
nFieldId	Nummer der Schaltfläche in der Werkzeugleiste
nCommandIndex	der ausgewählte Befehl aus dem Menü, wobei die Befehle von 0 weggezählt werden

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

long MToolbarMenuCommand (short nWinId, short nFieldId, short CommandIndex):

Das Kommando wird aufgezeichnet, wenn der Benutzer in der Werkzeugleiste eines Fensters einen der Menüpunkte aufruft, der von der gedrückten Schaltfläche angezeigt worden ist. Im Gegensatz zum MMessageDynamicMenuCommand - Befehl beruht dieses Menü auf einer anderen Technologie und wird intern anders verarbeitet. Im Makro wird an dieser Stelle immer zuerst ein MPushbutton - Befehl für diese Schaltfläche erzeugt (dieser Befehl füllt das Menü) und der MToolbarMenuCommand bei der Auswahl eines der Menüpunkte.

➤ **Parameter:**

NWinId	Nummer des Fensters
nFieldId	Nummer der Schaltfläche in der Werkzeugleiste
nCommandIndex	der ausgewählte Befehl aus dem Menü, wobei die Befehle von 0 weggezählt werden

➤ **Rückgabe:**

Ist abhängig von der Applikation, normalerweise 0

void MSavePreview (short nWinId, short nFieldId, BSTR strFilename, short nType):

Dieses Kommando wird nicht aufgezeichnet. Es kann vom Anwender in das Makro eingefügt werden, um eine Druckvorschau zu exportieren. Mit dem Parameter nType wird der Typ der exportierten Datei festgelegt, wobei alle Typen verwendet werden können, die auch beim manuellen Export über das Kontextmenü zur Verfügung stehen. Eine bestehende Datei wird kommentarlos überschrieben.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Vorschau befindet (wenn 0 dann wird die erste dargestellte Vorschau verwendet)
nFieldId	Nummer des Vorschau elements (in Standard Vorschau Fenstern immer 100) (wenn 0, wird 100 verwendet)
strFilename	Name der exportierten Datei

nType	Typ der exportierten Datei
0 ...	SPL - Datei
1 ...	MHT - Datei
2 ...	HTML
4 ...	SPL - Datei Version 2.0 (früheres Format)
5 ...	Acrobat PDF
6 ...	RTF Format (Microsoft Word)
7 ...	RTF Format
8 ...	Tabulatorgetrennter Text
9 ...	Text

➤ **Rückgabe:**

Keine

MPutIntoCampaign (short nWinId, short nFieldId, BOOL bAdd, BOOL bShowWindow, short RelateType, BSTR name):

Dieses Kommando wird aufgezeichnet wenn in einer Druckvorschau der Befehl ‚Zur Merkliste hinzufügen‘ gewählt wird, und im darauffolgenden Dialog eine Merkliste ausgewählt wird.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Vorschau befindet (wenn 0 dann wird ein geöffnetes Fenster mit einer Vorschau gesucht)
nFieldId	Nummer des Vorschauelements (in Standard Vorschau Fenstern immer 100) (wenn 0, wird 100 verwendet)
bAdd	ob die gefunden Objekte zu der gewählten Merkliste dazugefügt werden, oder diese ersetzen
bShowWindow	ob das Merklistenfenster sich öffnen soll, oder ob die Merkliste ohne das Fenster zu öffnen gespeichert werden soll.
RelateType	der Typ der Daten, die aus der Vorschau in die Merkliste übernommen werden sollen 0... alle möglichen Typen 1... Artikel 2... Arbeitnehmer 3... Projekte 4... CRM Fälle 5... Vertreter 6... Kontakte
Name	Der Name der Merkliste

➤ **Rückgabe:**

keine

VARIANT MGetLastUsedObjects (short Type):

Dieses Kommando wird nicht aufgezeichnet. Es kann vom Anwender in das Makro eingefügt werden, um das zuletzt verwendete globale Objekt verwenden zu können. Die globalen Objekte, die in einer Liste der letzten zehn verwendeten Objekte verwaltet werden, sind: Konten, Artikel, Projekte und Arbeitnehmer.

➤ **Parameter:**

Type	Typ des globalen Objekts, dessen Liste zurückgegeben werden soll
1...	Konten
41...	Produkte
30...	Projekte
91...	Arbeitnehmer

➤ **Rückgabe:**

Array mit den bis zu 10 zuletzt verwendeten Objekten

Beispiel

Im folgenden Beispiel werden im Makro die globalen Objekte aller vier Typen in einer MsgBox ausgegeben:

```
Sub RunMacro
    ' Your macro code
    Test = "Liste der zuletzt verwendeten Objekte:"
    Testarraccounts = Cwlmacro.Mgetlastusedobjects (1)
    Testarrproducts = Cwlmacro.Mgetlastusedobjects (41)
    Testarrprojects = Cwlmacro.Mgetlastusedobjects (30)
    Testarran = Cwlmacro.Mgetlastusedobjects (91)

    Accountlist = "Konten"
    For I = 0 To Ubound (Testarraccounts)
        Accountlist = Accountlist & Chr(13) & Chr (10) & Testarraccounts(I)
    Next

    Productlist = "Artikel"
    For I = 0 To Ubound (Testarrproducts)
        Productlist = Productlist & Chr(13) & Chr (10) & Testarrproducts(I)
    Next

    Projectlist = "Projekte"
    For I = 0 To Ubound (Testarrprojects)
        Projectlist = Projectlist & Chr(13) & Chr (10) & Testarrprojects(I)
    Next

    Anlist = "AN"
    For I = 0 To Ubound (Testarran)
        Anlist = Anlist & Chr(13) & Chr (10) & Testarran(I)
    Next
    MsgBox Accountlist & Chr(13) & Chr (10) & Productlist & Chr(13) & Chr (10) & Projectlist
    & Chr(13) & Chr (10) & Anlist
End Sub
```

MRunMacroSuspended (BSTR macroname, VARIANT params);

Dieses Kommando wird nicht aufgezeichnet. Es kann vom Anwender in das Makro eingefügt werden, um ein weiteres Makro zu starten, das aber nicht sofort ausgeführt wird, sondern erst wenn das aufrufende Makro aus dem aktuellen Aufruf zurückgekehrt ist. Ein Anwendungsfall wäre, aus einem MDP Fensterscript heraus das Fenster zu schließen in dessen Kontext das Script abläuft. Dies würde beim direkten Aufruf zu einer Ausnahme führen. Über den Umweg über ein weiteres Makro, aufgerufen mit diesem Kommando, kann dies durchgeführt werden.

➤ **Parameter:**

macroname	Der Name des aufgerufenen Makros
params	Ein Array mit Parametern, auf die das Aufgerufene Makro mit der MParameters – Funktion zugreifen kann.

➤ **Rückgabe:**

keine

MChooseFile (BSTR Filename, short ret);

Dieses Kommando wird aufgezeichnet wenn im Programm eine Dateiauswahl aufgerufen wird. Bei der Ausführung des Makros, wird der bei der Aufzeichnung gewählte Dateiname und der beim Aufruf der Originalfunktion zurückgegebene Ergebniswert verwendet.

➤ **Parameter:**

Filename	Der gewählte Dateiname mit Pfad
ret	der Ergebniswert des Originalaufrufs (0 bei Abbruch, 1 bei Auswahl einer Datei)

➤ **Rückgabe:**

keine

MDoModal (BSTR Value, short ret):

Dieses Kommando wird aufgezeichnet wenn im Programm ein modales Fenster aufgerufen wird. Das einzige mit diesem Aufruf unterstützte modale Fenster ist die modale Auswahl einer Grafik aus dem Grafikmatchcode (z.B. im Artikelstamm).

➤ **Parameter:**

Value	Der aktuelle Wert der globalen Variable, die bei Matchcode-Aufrufen den gewählten Wert enthält (im Falle des Grafikmatchcodes ist dies der Name der Grafik).
ret	der Ergebniswert des Originalaufrufs (0 bei Abbruch, 1 bei Beendigung des Fensters mit OK).

➤ **Rückgabe:**

keine

MExecDrillDown (short nWinId, short nFieldId, BSTR ItemText, BSTR HiddenText):

Dieses Kommando wird aufgezeichnet wenn in einer Druckvorschau ein Hyperlink angeklickt wird. Bei der Ausführung des Kommandos wird die gleiche Aktion durchgeführt, die der Klick auf den Hyperlink ausführen würde.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Vorschau befindet
nFieldId	Nummer des Vorschauelements (in Standard Vorschaufenstern immer 100) (wenn 0, wird 100 verwendet)
ItemText	der angezeigte Wert des angeklickten Hyperlinks
HiddenText	der versteckte Wert des Hyperlinks, der die beim Klick des Hyperlinks durchzuführende Aktion beschreibt.

➤ **Rückgabe:**

keine

MExecGridDrillDown (short nWinId, short nFieldId, int line, short col, BSTR ItemText, BSTR HiddenText):

Dieses Kommando wird aufgezeichnet wenn in einer Bildschirmtablette ein Hyperlink angeklickt wird. Bei der Ausführung des Kommandos wird die gleiche Aktion durchgeführt, die der Klick auf den Hyperlink ausführen würde.

➤ **Parameter:**

nWinId	Nummer des Fensters in dem sich die Bildschirmtablette befindet
nFieldId	Nummer der Bildschirmtablette
line	Zeile in der Bildschirmtablette
col	Spalte in der Bildschirmtablette
ItemText	der angezeigte Wert des angeklickten Hyperlinks
HiddenText	der versteckte Wert des Hyperlinks, der die beim Klick des Hyperlinks durchzuführende Aktion beschreibt.

➤ **Rückgabe:**

keine

MOpenGridAsXls (short nWinId, short nFieldId):

Dieses Kommando wird aufgezeichnet wenn in einer Bildschirmtable die 'Ausgabe Excel' - Schaltfläche gedrückt wird.

➤ Parameter:

nWinId Nummer des Fensters in dem sich die Bildschirmtable befindet
nFieldId Nummer der Bildschirmtable

➤ Rückgabe:

Keine

MGetRelativeDate (short nWhatDate):

Dieses Kommando wird nicht direkt aufgezeichnet, kann aber nach einer Aufzeichnung anstatt eines Datums eingegeben werden, damit beim Abspielen des Makros immer ein Datum relativ zum gerade aktiven Datum verwendet wird.

➤ Parameter:

nWhatDate Nummer des Datums, das beschreibt welches Datum relativ zum aktuellen verwendet werden soll:
0 oder 1 das aktuelle Datum
2 ein Tag vor dem aktuellen Tag
3 Anfang des Monats
4 Ende des Monats
5 Anfang des letzten Monats
6 Ende des letzten Monats
7 Anfang der Woche
8 Ende der Woche
9 Anfang der letzten Woche
10 Ende der letzten Woche

➤ Rückgabe:

Das Datum ohne Zeit.

MSaveFullGridSettings (short nWinId, short nFieldId, BSTR Setting):

Dieses Kommando wird aufgezeichnet wenn die Gesamteinstellungen einer Bildschirmtable gespeichert werden.

➤ Parameter:

nWinId Nummer des Fensters in dem sich die Bildschirmtable befindet
nFieldId Nummer der Bildschirmtable
Setting der Name der gespeicherten Einstellungen. Der Name ist leer wenn die <Standardeinstellungen> gespeichert werden.

➤ Rückgabe:

TRUE/FALSE

MLoadFullGridSettings (short nWinId, short nFieldId, BSTR Setting):

Dieses Kommando wird aufgezeichnet wenn in einer Bildschirmtable die Gesamteinstellungen geladen werden.

➤ Parameter:

nWinId Nummer des Fensters in dem sich die Bildschirmtable befindet
nFieldId Nummer der Bildschirmtable

Setting der Name der geladenen Einstellungen. Der Name ist leer wenn die <Standardeinstellungen> geladen werden.

➤ **Rückgabe:**
TRUE/FALSE

MPreviewButton (short nWinId, short nFieldId, short ButtonId, short AddParam):

Dieses Kommando wird aufgezeichnet wenn in einer Druckvorschau der PowerReport - Button ausgewählt wird.

➤ **Parameter:**
nWinId Nummer des Fensters der Druckvorschau
nFieldId Nummer des Druckvorschauelements im Fenster (100)
ButtonId Die interne Id des PowerReport Buttons (15022)
AddParam Ein zusätzlicher Parameter, der immer mit 1 übergeben wird

➤ **Rückgabe:**
Keine

2.1.3. Ereignisse

void OnRunMacro();

Dieses Kommando wird nicht aufgezeichnet. Das Ereignis wird ausgelöst (= die Funktion im Makro aufgerufen), wenn das Makro startet.

void OnStopMacro();

Dieses Kommando wird nicht aufgezeichnet. Das Ereignis wird ausgelöst (= die Funktion im Makro aufgerufen), wenn das Makro beendet wird.

2.2. Verwaltung von Makros

Die Verwaltung der Makros erfolgt im Programm WinLine START im Menüpunkt

 **Parameter**
 **Programm Makros**

Dort können neue Makros angelegt, bestehende Makros verändert oder gelöscht und auch Makros gestartet werden. Über dieses Fenster können allerdings keine Makros aufgezeichnet werden.

Das Fenster ist in drei Register verteilt, wobei ohne zusätzliche Lizenz nur das Register "Makros" zur Verfügung steht.

Die Register "Fenster Skripten" und "System Skripten" können nur dann angewählt werden, wenn die entsprechende MDP-RunTime-Lizenz zur Verfügung steht.

Im Register "Makros" werden alle Makros, die bereits erfasst wurden, angezeigt.

➤ **Starten**
Durch Anklicken des Starten-Buttons wird das gerade aktive Makro (dieses ist gelb hinterlegt) ausgeführt.

➤ **Editieren**

Durch Anklicken des Editieren-Buttons kann der Inhalt des Makros angesehen und auch verändert werden.

➤ **Löschen**

Durch Anklicken des Löschen-Buttons wird das aktuell ausgewählte Makro gelöscht.

➤ **Exportieren**

Durch Anklicken des Exportieren-Buttons kann das gerade aktive Makro (dieses ist gelb hinterlegt) in eine Textdatei exportiert werden. Diese Datei bekommt die Erweiterung MMR.

➤ **Importieren**

Durch Anklicken des Importieren-Buttons kann ein Makro (auch mehrere Makros) aus einer Textdatei importiert werden (ist auch mittels Drag & Drop möglich). Damit können Makros von einer Installation zu einer anderen transferiert werden.

➤ **Ende**

Durch Drücken der ESC-Taste wird das Fenster geschlossen.

➤ **Ohne Fenster ausführen**

Wird diese Option aktiviert, dann werden alle Makros im "Silent-Mode" ausgeführt, d.h. es werden keine Meldungen ausgegeben, die ggf. beim Aufnehmen des Makros angezeigt wurden. Das bewirkt auch, dass die einzelnen Fenster nur kurz geöffnet und wieder geschlossen werden und somit eine "zuckenden Effekt" auslösen.

2.3. Was kann mit Makros gemacht werden?

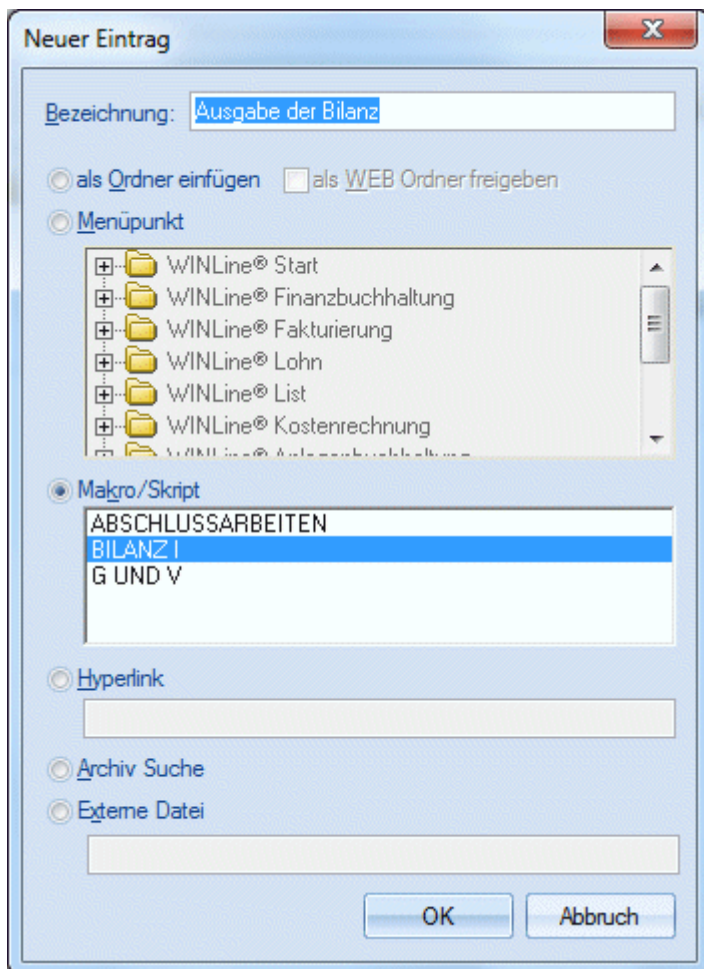
Makros können auf 4 verschiedene Arten aufgerufen und abgearbeitet werden:

- ☐ Aus der Buttonleiste bzw. aus dem Menüpunkt *Parameter/Programm Makros* im WinLine START. Diese Vorgangsweise wurde bereits in den vorherigen Kapiteln dargestellt.
- ☐ Aus den Favoriten
- ☐ Aus einer Befehlszeile z.B. einer Batchdatei oder einem Icon, wobei dem WinLine - Programm einige Parameter mitgegeben werden können.
- ☐ Aus der Funktion externe Programme (z.B. über die Buttonleiste Tools)
- ☐ Aus dem Cockpit, wo das Makro eingebunden und mit einen einfachen Klick gestartet werden kann.

2.3.1. Starten der Makros aus den Favoriten

Damit Makros aus den Favoriten gestartet werden können, ist folgende Vorgangsweise einzuhalten.

- ☐ Aufnahme und abspeichern des gewünschten Makros.
- ☐ Aufruf der Favoriten (rechte Maustaste in der Buttonleiste, Option Favoriten).
- ☐ Das Fenster "Favoriten" aktivieren und dort wieder mit der rechten Maustaste die Option "Neuer Eintrag" auswählen.



- ☐ Im Feld "Bezeichnung" wird der Text eingetragen, unter dem man das Makro aufrufen möchte. Als Option wird "Makro/Skript" eingestellt. Daraufhin kann aus der Listbox eines der angelegten Makros ausgewählt werden. Durch Anklicken des OK-Buttons wird der Eintrag in die Favoriten eingefügt.

2.3.2. Starten des Makros mit dem Programm

Mit dieser Option kann z.B. ein Sicherungslauf in der WinLine automatisch zu einer bestimmten Zeit durchgeführt werden. Dabei ist wieder folgende Vorgangsweise einzuhalten:

- ☐ Aufnahme und abspeichern des Makros. Dabei ist zu beachten, dass das Makro vom Schritt nach der Mandantenbestätigung (sofern vorhanden) aufgezeichnet werden muss.
- ☐ Einrichten der Befehlszeile, wobei folgende Parameter mitgegeben werden müssen:

/USERX

X steht für den Benutzer, mit dem das Makro ablaufen soll.

/PASSWDY

Y steht für das Password, das der Benutzer verwendet.

/COMPANYZ

Z steht für die Mandantennummer, mit der das Makro abgearbeitet werden soll. Diese Option muss nur dann mitgegeben werden, wenn das Programm WinLine START aufgerufen werden soll.

/YEARXXXX

XXXX steht für das Wirtschaftsjahr, das im Mandanten geöffnet werden soll. Das Wirtschaftsjahr muss so geschrieben werden, wie es auch in der Auswahllistbox des Mandantenwechsels angezeigt wird (z.B. /YEAR2003(10)). Dieser Parameter kann Optional verwendet werden. Wird der Parameter nicht gesetzt, wird der Mandant mit dem letzten WJ (Standard) gestartet.

/MACROMAKRO

MAKRO steht für den Makronamen, das durchgeführt werden soll.

/QUITAFTERMACRO

Mit diesem Parameter wird das Programm, nachdem das Makro abgearbeitet wurde, wieder beendet.

Beispiel:

So sieht die Befehlszeile aus, wenn ein Datensicherungsmakro automatisch gestartet werden soll:

C:\WinLine\ADMN.EXE /USERa /PASSWD b /MACROSICHERN /QUITAFTERMACRO

Dadurch wird das Programm WinLine ADMIN gestartet, der Benutzer a loggt sich mit Password b ein und danach wird sofort das Makro "Sichern" ausgeführt. Nachdem das Makro ausgeführt wurde, wird auch das Programm WinLine ADMIN wieder beendet.

2.3.3. Starten des Makros aus den externen Programmen

Makros können bei den externen Programmen unter dem Menüpunkt

 **Applikationen**
 **externe Programme**
hinterlegt werden.

In der Spalte, wo man normalerweise das Verzeichnis des Programms einträgt, wird bei einem Makro folgendes eingetragen:

MACRO:XXX (XXX steht für den Namen des Makros).

Das Makro kann nun entweder über den Menüpunkt

 **Applikationen**
 **Makroname**

oder über die Buttonleiste "Tools" aufgerufen werden.

2.3.4. Starten des Makros aus dem Cockpit

Damit ein Makro im Cockpit hinterlegt werden kann, sind folgende Schritte durchzuführen:

Zuerst muss das Cockpit ausgewählt werden, in dem das Makro hinterlegt werden soll. Dort wird dann auf den "Bearbeiten"-Link geklickt, wodurch die Bearbeitung möglich wird. Hier kann nun ein neuer Eintrag erfasst werden, wobei die Option "Makro" verwendet werden muss.



Folgende Einstellungen können vorgenommen werden:

➤ **Name**

Eingabe des Namens, der in weiterer Folge im Cockpit angezeigt werden soll. Der Name wird aus dem gewählten Makro übernommen und kann nachträglich noch verändert werden

Im unteren Teil des Fensters werden alle Makros angezeigt, die aufgenommen wurden. Aus diesen Einträgen kann der gewünschte ausgewählt werden.

Der Eintrag kann durch Drücken der F5-Taste oder durch Anklicken des VOR-Buttons gespeichert werden.

Im Cockpit wird in der entsprechenden Rubrik ein zusätzlicher Eintrag dargestellt, wobei dieser mit dem Icon  dargestellt wird. Wird dieser Eintrag angeklickt, wird das hinterlegte Makro aufgerufen und abgearbeitet.

Hinweis:

In diesem Zusammenhang ist es wichtig, dass die Makros so aufgenommen werden, dass sie immer vom Programm WinLine START aus abgearbeitet werden können. D.h. im Makro sollte auch der Wechsel vom WinLine START in die entsprechende Applikation mit aufgenommen werden, damit der Aufruf auch vom Cockpit aus funktioniert. Fehlt der Aufruf der Applikation, dann wird das Makro entweder falsch abgearbeitet (falscher Menüpunkt wird aufgerufen) oder scheitert bereits am Menüaufruf.